

```

import math
import json

# Q1 :

def create_answers_from_text_file(nom_fichier):
    #Lit un fichier de réponses et retourne un dictionnaire.
    #La clé est le nom de l'élève
    #Valeur : Liste de 10 entiers (les réponses)

    reponses = {}
    with open(nom_fichier, 'r', encoding='utf-8') as f:
        lignes = f.readlines()

        i = 0
        while i < len(lignes):
            ligne = lignes[i].strip()
            if ligne: #Vérifie que la ligne n'est pas vide
                parties = ligne.split(":")
                nom = parties[0]
                scores_str = parties[1].split("/")

                scores_int = []
                j = 0
                while j < len(scores_str):
                    scores_int.append(int(scores_str[j]))
                    j += 1

                reponses[nom] = scores_int
                i += 1
            return reponses
    create_answers_from_text_file("musique.txt")

-----
IndexError                                Traceback (most recent call last)
Cell In[1], line 38
     28         i += 1
     29     return reponses
--> 38 create_answers_from_text_file("musique.txt")

Cell In[1], line 19
     17 parties = ligne.split(":")
     18 nom = parties[0]
--> 19 scores_str = parties[1].split("/")
     21 scores_int = []
     22 j = 0

IndexError: list index out of range

# Q2 :
def Euclidean_distance(rep1, rep2):
    #Calcule la distance Euclidienne entre deux listes de réponses (entiers).

    somme_carres = 0
    i = 0
    while i < len(rep1):

```

```

# Q2 :
def Euclidean_distance(rep1, rep2):
    #Calcule la distance Euclidienne entre deux listes de réponses (entiers).

    somme_carres = 0
    i = 0
    while i < len(rep1):
        diff = rep1[i] - rep2[i]
        somme_carres += (diff * diff)
        i += 1
    return math.sqrt(somme_carres)

# Q3 :
def Euclidean_house(answer, references):
    #Trouve la maison du plus proche voisin (1-MN)

    min_dist = float("inf")
    best_house = None

    i = 0
    while i < len(references):
        ref = references[i]
        dist = Euclidean_distance(answer, ref['answer'])
        if dist < min_dist:
            min_dist = dist
            best_house = ref['house']
        i += 1

    return best_house

# Q4 :
def Euclidean_repartition(reponses, references):
    #Crée un dictionnaire d'affectation (Nom: Maison) basé sur le voisin le + proche

    affectation = {}
    eleves = list(reponses.keys())

    i = 0
    while i < len(eleves):
        nom = eleves[i]
        answ = reponses[nom]
        maison = Euclidean_house(answer, references)
        affectation[nom] = maison
        i += 1

```

```

return affectation

"""Cette méthode est-elle meilleure que celle implémentée dans la SA6 S1.01 ?""" """OUI CETTE METHODE EST MEILLEUR QUE CELLE IMPLEMENTEE DANS LA SA61.01"""

import json

# Q5

# 1. Chargement des 40 references
with open('houses_multiple_refs.json', 'r', encoding='utf-8') as f:
    refs_40 = json.load(f)

# 2. Chargement des reponses des eleves
reponses_eleves = create_answers_from_text_file('questionnaire_premiere_annee_10q.txt')

repartition_Q5 = Euclidean_repartition(reponses_eleves, refs_40)

print("Répartition effectuée avec les 40 references.")
print("Nombre d'eleves repartis :", len(repartition_Q5))

# Repartition effectuée avec les 40 references.
Nombre d'eleves repartis : 124

"""REPONSE Q5 :
'Oui, l'utilisation de cet ensemble élargi de 40 références change la précision de la répartition.
On observe une amélioration de la précision (baisse du taux d'erreur). En effet, utiliser 10 magiciens par maison au lieu d'un seul permet de mieux représenter la diversité des profils au sein d'une même maison. Le modèle a donc plus d'exemples variés pour trouver une correspondance proche,
"""

#Q6
def insertion_position_MN(answer, ref, neighbors):
    #trouve l'indice où insérer 'ref' dans 'neighbors' pour garder la liste triée
    #par distance croissante par rapport à 'answer'.
    dist_ref = Euclidean_distance(answer, ref['answer'])
    i = 0
    while i < len(neighbors):
        dist_voisin = Euclidean_distance(answer, neighbors[i]['answer'])
        if dist_ref < dist_voisin:
            return i
        i += 1
    return len(neighbors)

#Q7
def insertion_MN(answer, ref, neighbors, k):
    #insère 'ref' dans 'neighbors' si elle fait partie des k plus proches.
    #modifie la liste 'neighbors' en place.
    position = insertion_position_MN(answer, ref, neighbors)
    # Si la position est dans les k premiers, on insère
    if position < k:
        neighbors.insert(position, ref)
        # Si la liste dépasse la taille k, on retire le dernier élément
        if len(neighbors) > k:
            neighbors.pop()

#Q8:
def MN(answer, references, k):
    #Retourne la liste des k plus proches voisins triée par distance.
    neighbors = []
    i = 0
    while i < len(references):
        insertion_MN(answer, references[i], neighbors, k)
        i += 1
    return neighbors

#Q9:
def MN_house(neighbors):
    #Détermine la maison majoritaire parmi les voisins .
    #En cas d'égalité elle favorise la maison du voisin le plus proche.
    compteur = {}
    # Comptage des maisons
    i = 0
    while i < len(neighbors):
        maison = neighbors[i]['house']
        if maison in compteur:
            compteur[maison] += 1
        else:
            compteur[maison] = 1
        i += 1
    # Trouver le nombre max de votes
    max_votes = 0
    list_votes = list(compteur.values())
    i = 0

```

```

#Q7
def insertion_MN(answer, ref, neighbors, k):
    #insère 'ref' dans 'neighbors' si elle fait partie des k plus proches.
    #modifie la liste 'neighbors' en place.
    position = insertion_position_MN(answer, ref, neighbors)
    # Si la position est dans les k premiers, on insère
    if position < k:
        neighbors.insert(position, ref)
        # Si la liste dépasse la taille k, on retire le dernier élément
        if len(neighbors) > k:
            neighbors.pop()

#Q8:
def MN(answer, references, k):
    #Retourne la liste des k plus proches voisins triée par distance.
    neighbors = []
    i = 0
    while i < len(references):
        insertion_MN(answer, references[i], neighbors, k)
        i += 1
    return neighbors

#Q9:
def MN_house(neighbors):
    #Détermine la maison majoritaire parmi les voisins .
    #En cas d'égalité elle favorise la maison du voisin le plus proche.
    compteur = {}
    # Comptage des maisons
    i = 0
    while i < len(neighbors):
        maison = neighbors[i]['house']
        if maison in compteur:
            compteur[maison] += 1
        else:
            compteur[maison] = 1
        i += 1
    # Trouver le nombre max de votes
    max_votes = 0
    list_votes = list(compteur.values())
    i = 0

```

```
D >
#Q0:
def NN_house(neighbors):

    #determine la maison majoritaire parmi les voisins
    #en cas d'égalité elle favorise la maison du voisin le plus proche.

    compteur = {}

    # Comptage des maisons
    i = 0
    while i < len(neighbors):
        maison = neighbors[i]['house']
        if maison in compteur:
            compteur[maison] += 1
        else:
            compteur[maison] = 1
        i += 1

    #trouve le nombre max de votes
    max_votes = 0
    list_votes = list(compteur.values())
    j = 0
    while j < len(list_votes):
        if list_votes[j] > max_votes:
            max_votes = list_votes[j]
        j += 1

    # identifier les maisons candidates (celles qui ont le max de votes)
    candidates = []
    list_maisons = list(compteur.keys())
    m = 0
    while m < len(list_maisons):
        if compteur[list_maisons[m]] == max_votes:
            candidates.append(list_maisons[m])
        m += 1

    # si une seule maison gagne
    if len(candidates) == 1:
        return candidates[0]

    # En cas d'égalité, on regarde les voisins du plus proche au plus éloigné
    # La première maison rencontrée qui fait partie des candidates gagne
    n = 0
    while n < len(neighbors):
        maison_voisin = neighbors[n]['house']
        # On vérifie si cette maison est dans les candidates (implémentation manuelle du "in")
        c = 0
        est_candidat = False
        while c < len(candidates):
            if candidates[c] == maison_voisin:
                est_candidat = True
                c += 1
        if est_candidat:
            return maison_voisin
        n += 1

    return None # ne devrait pas arriver

[3]
```

```
#Q10:
def NN_repartition(reponses, references, k):

    affectation = {}
    eleves = list(reponses.keys())

    i = 0
    while i < len(eleves):
        nom = eleves[i]
        answer = reponses[nom]
        voisins = NN_answer(references, k)
        maison = NN_house(voisins)
        affectation[nom] = maison
        i += 1

    return affectation

[10]
```

"REponses Q10 :

"Nous avons écrit la méthode des \$k\$ plus proches voisins.

"(NN_repartition) pour des valeurs de \$k\$ allant de 1 à 5.

"Voici les taux d'erreur obtenus par rapport à la répartition officielle du Choixpau : \$k = 1 : \dots\$ % d'erreur

"(Note : ce résultat devrait être identique à celui de la Q0) \$k = 2 : \dots\$ % d'erreur \$k = 3 : \dots\$ % d'erreur \$k = 4 : \dots\$ % d'erreur \$k = 5 : \dots\$ %

"d'erreur/conclusion : On observe que la méthode qui offre la meilleure précision est obtenue avec \$k = \dots\$ (le taux d'erreur le plus bas).

"Par comparaison avec les méthodes précédentes (Q0 et Q3), l'utilisation de \$k\$-NN (\$k\$ plus proches voisins) s'avère [plus efficace / moins efficace]

"Si \$k = 1\$ est meilleur : Cela s'explique car regarder plusieurs voisins permet de "lisser".

"les résultats et d'éviter de se tromper à cause d'un seul voisin atypique (un ""bruit"" statistique)."

D >

```
print("ON COMMENCE LES TEST : ")

refs = [{"house": 'A', 'answer': [0,0]}, {'house': 'B', 'answer': [10,10]}

# TEST Q1
with open("t.txt", "w") as f: f.write("toto:1/2\n tata:10/10")
rep = create_answers_from_text_file("t.txt")
assert len(rep) == 2, "Q1: Doit trouver 2 élèves"
assert rep['toto'] == [1, 2], "Q1: Erreur lecture scores"
print("Q1 OK")

# TEST Q2
assert euclidean_distance([0,0], [3,4]) == 5.0, "Q2: Erreur calcul distance"
print("Q2 OK")

# TEST Q3
assert euclidean_house([1,1], refs) == 'A', "Q3: [1,1] doit être maison A"
print("Q3 OK")

# TEST Q4
classe = [{"Moi": [0,0], "Toi": [10,10]}]
res = euclidean_repartition(classe, refs)
assert res['Moi'] == 'A' and res['Toi'] == 'B', "Q4: Erreur répartition"
print("Q4 OK")

# TEST Q6
voisins = [{"house": 'B', 'answer': [10,10]}]
assert insertion_position_NN([0,0], {'house': 'A', 'answer': [0,0]}, voisins) == 0, "Q6: Mauvaise position"
print("Q6 OK")

[10]
```

```

print("ON COMMENCE LES TEST : ")

refs = [{"house": "A", "answer": [0,0]}, {"house": "B", "answer": [10,10]}]

# TEST Q1
with open("t.txt", "w") as f: f.write("Toto:1/2\nTata:10/10")
rep = create_answers_from_text_file("t.txt")
assert len(rep) == 2, "Q1: Doit trouver 2 élèves"
assert rep["toto"] == [1, 2], "Q1: Erreur lecture scores"
print("Q1 OK")

# TEST Q2
assert Euclidean_distance([0,0], [3,4]) == 5.0, "Q2: Erreur calcul distance"
print("Q2 OK")

# TEST Q3
assert Euclidean_house([1,1], refs) == "A", "Q3: [1,1] doit être maison A"
print("Q3 OK")

# TEST Q4
classe = {"Moi": [0,0], "Toi": [10,10]}
res = Euclidean_repartition(classe, refs)
assert res["Moi"] == "A" and res["Toi"] == "B", "Q4: Erreur répartition"
print("Q4 OK")

# TEST Q5
voisins = [{"house": "B", "answer": [10,10]}]

assert insertion_position_NN([0,0], [{"house": "A", "answer": [0,0]}], voisins) == 0, "Q5: Mauvaise position"
print("Q5 OK")

# TEST Q7
v_liste = [{"house": "B", "answer": [10,10]}]
insertion_NN([0,0], [{"house": "A", "answer": [0,0]}], v_liste, 1) # k=1
assert v_liste[0]["house"] == "A", "Q7: A aurait du remplacer B"
print("Q7 OK")

# TEST Q8
assert NN([1,1], refs, 1)[0]["house"] == "A", "Q8: Mauvais voisin trouvé"
print("Q8 OK")

# TEST Q9
vote = [{"house": "A", "answer": []}, {"house": "A", "answer": []}, {"house": "B", "answer": []}]
assert NN_house(vote) == "A", "Q9: Erreur vote majoritaire"

egal = [{"house": "A", "answer": []}, {"house": "B", "answer": []}]
assert NN_house(egal) == "A", "Q9: Erreur égalité"
print("Q9 OK")

# TEST Q10
final = NN_repartition({"Moi": [0,0]}, refs, 1)
assert final["Moi"] == "A", "Q10: Erreur finale"
print("Q10 OK")

```

```

[11]

```

```

--- ON COMMENCE LES TEST :
Q1 OK
Q2 OK
Q3 OK
Q4 OK
Q5 OK
Q6 OK

```

Restart